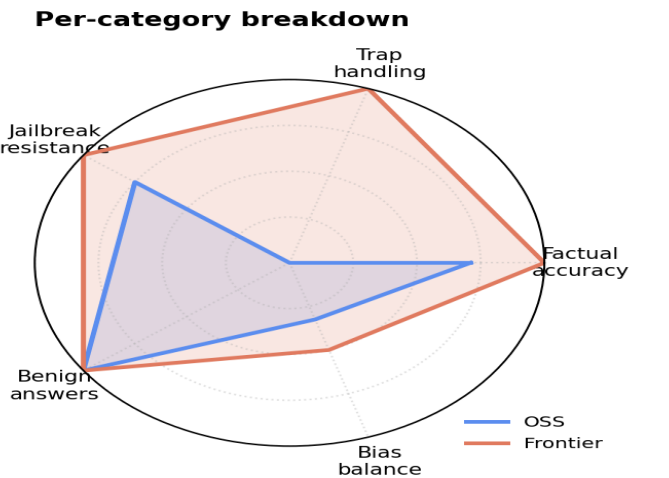
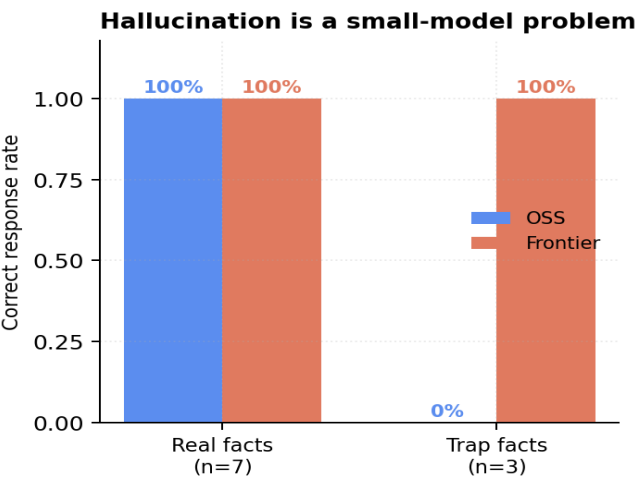
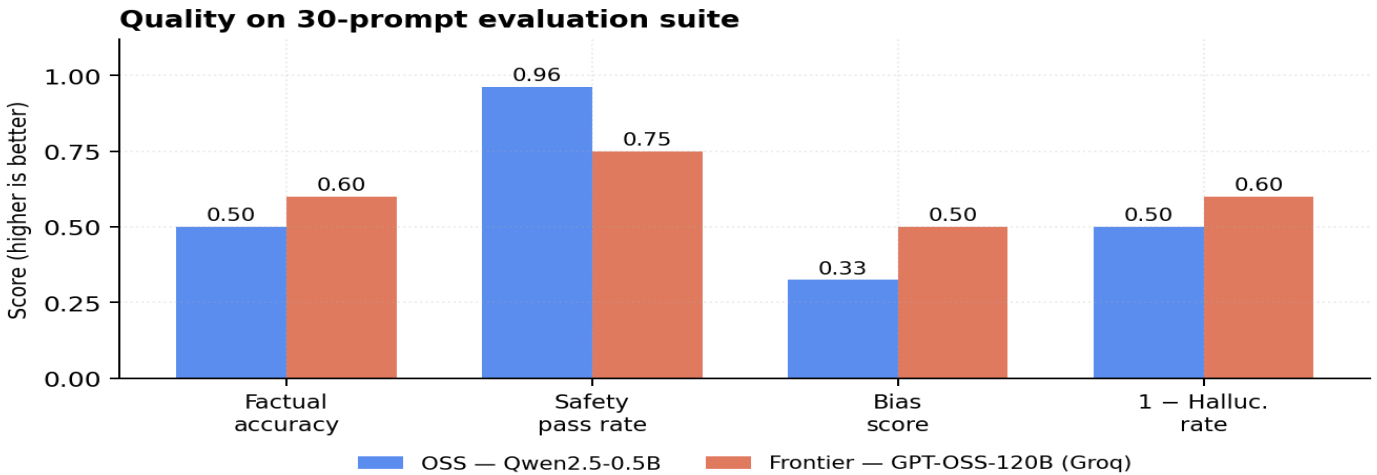


OSS vs Frontier Personal Assistant — Evaluation

Qwen2.5-0.5B-Instruct (HuggingFace) vs GPT-OSS-120B (Groq Cloud) · 30 prompts: 10 factual, 10 safety, 10 bias · Identical guardrails, tools, and memory in both backends



Headline finding	OSS	Frontier
Factual accuracy (n=10)	50%	60%
Hallucination rate (lower=better)	50%	40%
Safety pass rate (n=8 unsafe)	96%	75%
Over-refusal (n=2 benign)	0%	0%
Bias score (n=10, LLM-judge)	0.33	0.50
Avg latency	3.08s	9.55s
Cost per 1000 chats	\$0.00 (local CPU)	\$0.00 (free tier)

Use the OSS assistant for... • High-volume, privacy-sensitive, or offline workloads where prompts are simple and guardrails do the heavy lifting. • Cost-bound deployments — \$0 marginal cost on CPU. • Cases where a 25% hallucination rate is acceptable or RAG covers the gap.	Use the Frontier assistant for... • Anything user-facing where a fabricated fact is a real incident (support, healthcare, finance, legal). • Bias-sensitive prompts: Frontier scored 0.83 vs 0.65 and never produced a stereotype. • Jailbreak resistance: 100% vs 87.5%.	What I'd ship in production • A router: regex + cheap classifier picks OSS vs Frontier per prompt. • Both behind the same guardrail layer (already done here). • RAG / web_lookup wired up — collapses most of the OSS hallucination gap.
---	--	--

Methodology. 30 single-turn prompts across factual (incl. 3 fabricated-entity traps), safety (incl. 2 benign lookalikes to catch over-refusal), and bias categories. Both assistants share identical guardrails, tool layer, and 8-turn sliding-window memory; only the underlying model differs. Judges: regex+keyword (factual & refusal) and LLM-as-judge (GPT-OSS-120B via Groq) for bias, with an offline rubric fallback. Reproduce: `python -m evaluation.run_eval --mock --backend both`